

# ADAPTIVE VR LEARNING SYSTEM

Technical handoff: completed work, operating workflow, verification status, and remaining roadmap

Project owner: Harsh Singh

Repository target: HarshSingh0831/VR\_Software

Prepared: 22 July 2026

# 1. Executive summary

This project is an adaptive learning system for a smartphone-based VR headset. It combines a DC motor lesson, eight short supporting videos, bilingual continuous voice commands, upper-face camera inference, timed quizzes, session logging, and rule-based multimodal engagement analysis.

## Current outcome

**The laptop and smartphone can open the same HTTPS Streamlit dashboard simultaneously. A shared in-process synchronization bus distributes content selection, playback commands, quiz phase, and understanding-check state between active browser sessions. The smartphone browser supplies microphone commands; the Raspberry Pi and ESP32 camera pipeline supplies the upper-face signal.**

## Important portability note

Large MP4 lesson files, trained model weights, runtime recordings, private keys, certificates, Wi-Fi credentials, and Raspberry Pi secrets are intentionally excluded from Git. A new machine must restore those local assets and regenerate its HTTPS certificate.

# 2. System architecture

| Layer              | Technology                                      | Responsibility                                                                                                 |
|--------------------|-------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| VR learner         | Smartphone browser + headset                    | Displays lesson, subcontent, quizzes, voice status, and analysis.                                              |
| Dashboard          | Python 3 + Streamlit + Custom Components v2     | Responsive interface, video control, popup timing, quiz workflow, and synchronization.                         |
| Upper-face sensing | DFRobot ESP32-S3 AI camera                      | Captures the visible eye/forehead region in grayscale-compatible frames.                                       |
| Edge gateway       | Raspberry Pi 4 Model B                          | Receives camera data, exposes preview/status, stores labeled recordings, and connects to laptop.               |
| Inference          | CNN inference module                            | Produces a live upper-face expression estimate; target states are focused, confused, happy, bored, and drowsy. |
| Speech             | Vosk laptop service + browser SpeechRecognition | English, Hindi, and Hinglish commands; confusion, help, repeat, and hesitation features.                       |
| Analytics          | JSONL session events + rule baseline            | Combines camera emotion, speech behaviour, quiz correctness, and response time.                                |

# 3. Implemented learner workflow

## Main lesson

The 9:01 DC motor video is played on the laptop or smartphone. Playback commands are synchronized across active dashboard sessions.

## Timed suggestions

A compact subcontent name appears in the upper-left corner for 30 seconds at its configured timestamp. Only one suggestion is shown at a time.

## Voice selection

The learner can say Play Torque, Play Current Reverse, Play Multiple Coils, Play Commutator, Play Brushes, Play Magnets, or Play Electromagnet.

## Understanding check

After a subcontent video ends, the app asks whether the learner understood. Yes resumes the main video from its stored browser position; No restarts the explanation.

## Quiz schedule

Seven MID PART 1 questions open at approximately 4:30. Seven LAST PART questions open when the main lesson finishes.

## Adaptation signal

Support is recommended when the camera indicates a support-needed state, the learner uses confusion/help/repeat language, or quiz accuracy falls below the rule threshold.

## 4. Subcontent schedule

| Popup time | Subcontent      | Popup      | Media status |
|------------|-----------------|------------|--------------|
| 1:13       | Magnets         | 30 seconds | Integrated   |
| 2:35       | Electromagnet   | 30 seconds | Integrated   |
| 4:20       | Curved Magnet   | 30 seconds | Integrated   |
| 4:50       | Commutator      | 30 seconds | Integrated   |
| 4:59       | Brushes         | 30 seconds | Integrated   |
| 5:35       | Current Reverse | 30 seconds | Integrated   |
| 6:14       | Multiple Coils  | 30 seconds | Integrated   |
| 7:02       | Torque          | 30 seconds | Integrated   |

The timestamps are lesson positions taken from the supplied WhatsApp schedule, not the durations of the short subcontent files.

## 5. Quiz content

Two supplied Word documents were parsed into config/learning\_quiz.json. The valid LAST PART copy was DC\_Motor\_LAST\_PART\_Quiz (2).docx; the same filename without (2) was a zero-byte duplicate.

| Phase    | Trigger                       | Questions | Coverage                                                                       |
|----------|-------------------------------|-----------|--------------------------------------------------------------------------------|
| Midpoint | Main video reaches about 4:30 | 7         | Circuit, magnetic force, armature, torque, commutator, energy conversion       |
| Final    | Main video ends               | 7         | Brushes, curved poles, multiple coils, components, current path, smooth torque |

## 6. Files a future Codex session should inspect first

| File                              | Purpose                                                                  |
|-----------------------------------|--------------------------------------------------------------------------|
| learning_app.py                   | Primary smartphone/laptop learning dashboard and interaction workflow.   |
| config/learning_content.json      | Main lesson, eight subcontent paths, popup timestamps, and availability. |
| config/learning_quiz.json         | Fourteen MID and LAST quiz questions with answer keys.                   |
| host/adaptive_vr/cnn_inference.py | Live upper-camera model loading and prediction.                          |
| host/adaptive_vr/speech_live.py   | Continuous offline laptop speech recognition service.                    |
| host/adaptive_vr/dashboard_pi.py  | Raspberry Pi status, preview, and recording control.                     |
| host/adaptive_vr/learning_log.py  | Session event persistence and summary metrics.                           |
| firmware/esp32_local_processor/   | ESP32-S3 camera acquisition and local-processing firmware.               |
| raspberry-pi/                     | Pi receiver/service setup and camera integration resources.              |

## 7. Completed work

| Area                       | Status                     | Evidence / behaviour                                                       |
|----------------------------|----------------------------|----------------------------------------------------------------------------|
| Architecture and protocols | Ready                      | ESP32, Pi, laptop, browser, analytics, and Unity message structures exist. |
| Eight subcontent videos    | Ready locally              | All eight files are mapped; media remains on D: and is excluded from Git.  |
| Timed corner popup         | Ready                      | Left-corner name, 30-second visibility, timestamp schedule.                |
| Voice video selection      | Ready                      | English/Hindi/Hinglish command routing and named subcontent selection.     |
| Understanding verification | Ready                      | Automatic end prompt, replay or main-video continuation.                   |
| Two quiz sets              | Ready                      | 14 verified questions: 7 midpoint and 7 final.                             |
| Upper camera preview       | Ready when hardware online | Pi gateway and ESP32 connection monitoring are implemented.                |
| CNN inference integration  | Baseline ready             | Live prediction is connected to the dashboard.                             |
| Multimodal rule analysis   | Ready baseline             | Camera, speech, quiz accuracy, help and replay events are combined.        |
| Laptop + smartphone HTTPS  | Ready on current network   | Dashboard listens on 0.0.0.0:8502 using a local certificate.               |
| Automated verification     | Passing                    | 51 repository tests and Streamlit AppTest passed at handoff time.          |

## 8. Remaining work and priority

| Priority | Remaining item                                    | Acceptance condition                                                                                                       |
|----------|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| P0       | Collect balanced headset-specific upper-face data | Five target states have enough labeled samples across users, lighting, and sessions.                                       |
| P0       | Fine-tune and validate the five-state CNN         | Held-out participant evaluation, confusion matrix, per-class precision/recall/F1, and saved production weights.            |
| P0       | Hardware endurance test                           | ESP32/Pi/dashboard runs continuously without stream loss, overheating, or memory failure.                                  |
| P1       | Persistent multi-device synchronization           | Replace the in-process bus with SQLite/Redis/WebSocket state that survives server restarts and supports multiple learners. |
| P1       | Calibrate emotion thresholds                      | Rules are derived from real headset trials rather than generic pretrained facial labels.                                   |
| P1       | Speech evaluation                                 | English, Hindi, and Hinglish command accuracy is measured under headset noise.                                             |
| P1       | Unity/VR application completion                   | Production VR scene consumes live adaptation messages and is tested in the headset.                                        |
| P1       | Secure deployment                                 | Stable hostname, trusted certificate, authentication, credential rotation, and no hard-coded network address.              |
| P2       | Research evaluation                               | Ethics/consent, participant protocol, baseline comparison, statistical analysis, and final paper results.                  |

## 9. Setup on another computer or Codex account

1. Clone the repository and create a Python virtual environment from pyproject.toml.

2. Restore the Vosk models under models/ and the lesson/subcontent MP4 files under the configured local media path.
3. Create firmware/esp32\_local\_processor/secrets.h from the example file; never commit credentials.
4. Configure Raspberry Pi hostname/IP and SSH key locally.
5. Regenerate HTTPS certificates for the current laptop IP using scripts/generate\_local\_https\_cert.py, then install the CA certificate on the phone.
6. Run the continuous speech service and then run learning\_app.py with Streamlit on port 8502.
7. Open the HTTPS network URL on the smartphone and localhost/network URL on the laptop.
8. Run the test suite before collecting data or changing the inference model.

## 10. Recommended next Codex prompt

**Continue the Adaptive VR Learning System from output/pdf/Adaptive\_VR\_Project\_Handoff.pdf. First inspect learning\_app.py, config/learning\_content.json, config/learning\_quiz.json, and the current git status. Preserve the eight-video workflow, 14-question midpoint/final quiz, continuous bilingual speech, Raspberry Pi/ESP32 camera path, and secret exclusions. The next objective is to replace the temporary in-process device synchronization with persistent per-session state, then run a real headset data collection and five-state CNN fine-tuning evaluation. Do not claim hardware or model accuracy without a live verification result.**

End of handoff